

Session 13 : Les moteurs physiques commerciaux

Objectifs de la session

- Comprendre **pourquoi** on utilise un moteur physique existant en production plutôt que le sien.
- Découvrir les moteurs majeurs (**PhysX, Havok, Jolt, Box2D, Rapier**) et leurs cas d'usage.
- Découvrir les concepts qu'ils partagent : **CCD, sleeping, joints, query** (raycast).
- Comprendre le **déterminisme** et pourquoi il est difficile à obtenir.
- Introduire l'architecture **ECS** (Entity-Component-System) qui structure les moteurs modernes.
- **Pratique** : utiliser **Rapier** avec Three.js, et la physique intégrée de **Godot**.

💡 Le moment charnière du cours

Depuis la Session 1, vous construisez **votre propre moteur** : intégrateurs, collisions, impulsions, ressorts, particules. Vous savez maintenant **pourquoi c'est difficile**. Cette session répond à la question : **“Et en production, on refait tout ça ?”** — Non. On utilise un moteur éprouvé. Mais maintenant, vous **comprenez ce qu'il fait sous le capot**, et c'est ce qui distingue un programmeur de jeux qui **utilise** un moteur d'un programmeur qui le **comprend**.

Partie 1 : Pourquoi utiliser un moteur physique ?

Le problème de la complexité

Notre moteur “maison” couvre des cas simples : quelques dizaines d'objets, formes convexes, impulsions basiques. Un jeu réel exige :

- **Des centaines d'objets** en interaction simultanée (empilements, destructions).
- **Des formes complexes** : convexes, concaves (meshes), composées.
- **De la stabilité** : un empilement de cubes ne doit pas trembler ni exploser.
- **Des joints** : charnières, ressorts, moteurs (ragdolls, véhicules).
- **Des queries** : raycasts pour les tirs, shapecasts pour les personnages.
- **De la performance** : tout ça en moins de 2 ms par frame.

Avantages des moteurs existants

- **Complexité** : gestion des contacts multiples, stabilité des empilements, solveurs itératifs optimisés.
- **Temps** : des décennies de R&D cumulées (PhysX a plus de 20 ans).
- **Performance** : SIMD, multithreading, broad phase optimisée — des milliers d'objets à 60 FPS.
- **Outils** : debug rendering, serialization, détermination des scènes.
- **Communauté** : bugs connus, solutions documentées, intégrations prêtes.

💬 Mais pourquoi on a fait notre moteur alors ?

Parce qu'un moteur physique est une **boîte noire dangereuse** si on ne comprend pas ce qu'elle contient. Sans les Sessions 1–12, un moteur commercial est magique : **“pourquoi mon personnage traverse le sol ?”, “pourquoi mon empilement tremble ?”, “pourquoi mes objets flottent ?”**. Avec elles, vous savez que c'est une question de **timestep**, de **solver iterations**, de **sleeping threshold** — et vous savez **où regarder**.

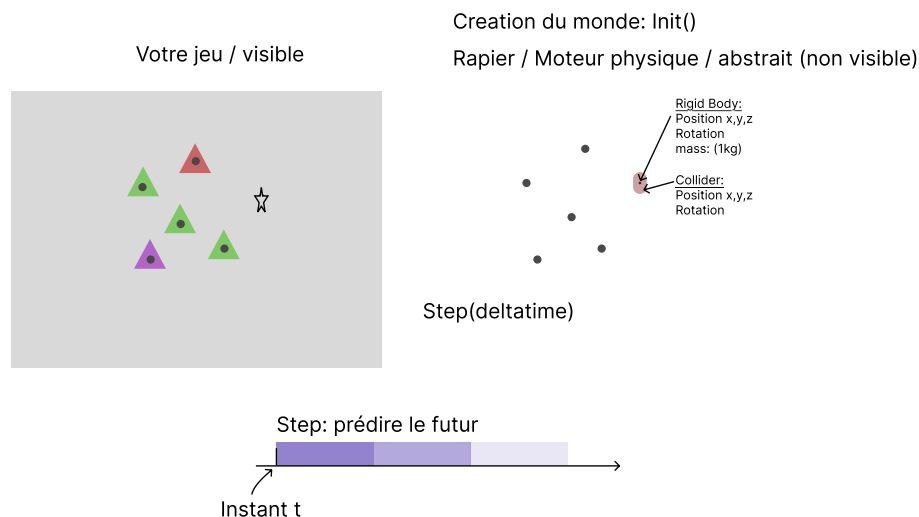


Figure 1: Un moteur physique commercial : on fournit les colliders et les forces, il retourne les positions et rotations. Toute la machinerie interne — broad phase, narrow phase, solveur de contraintes — est invisible mais fonctionne sur les principes vus en cours.

Partie 2 : Concepts partagés par tous les moteurs

Rigid Body vs Soft Body

Rigid Body vs Soft Body

- **Rigid Body** : objet **indéformable** (boîte, sphère, capsule). La distance entre deux points internes ne change jamais. C'est ce que tous les moteurs font nativement — et tout ce dont un jeu a besoin dans 95 % des cas.
- **Soft Body** : objet **déformable** (tissu, gelée, ballon). La forme change en fonction des forces appliquées. Beaucoup plus coûteux — souvent simulé par un système masse-ressort comme notre tissu de la Session 8, ou par des techniques spécialisées (FEM, position-based dynamics).

💡 La stratégie classique

Les jeux utilisent des **rigid bodies** presque partout, et simulent la déformation **visuellement** : un ragdoll est un ensemble de rigid bodies reliés par des joints ; une destruction est un échange d'un mesh statique contre plusieurs rigid bodies. La déformation "vraie" (soft body) est réservée aux cas où elle est le gameplay (World of Goo, JellyCar).

Continuous Collision Detection (CCD)

Le problème du tunneling

Un objet rapide peut **traverser** un mur mince entre deux frames. Si une balle va à 100 m/s avec un dt de 1/60 s, elle se déplace de **1,67 m par pas**. Un mur de 0,2 m d'épaisseur ? La balle est devant le mur à t , derrière à $t + dt$ — aucune des deux positions n'est **dans** le mur. Le test discret (Session 6) ne voit jamais la collision.

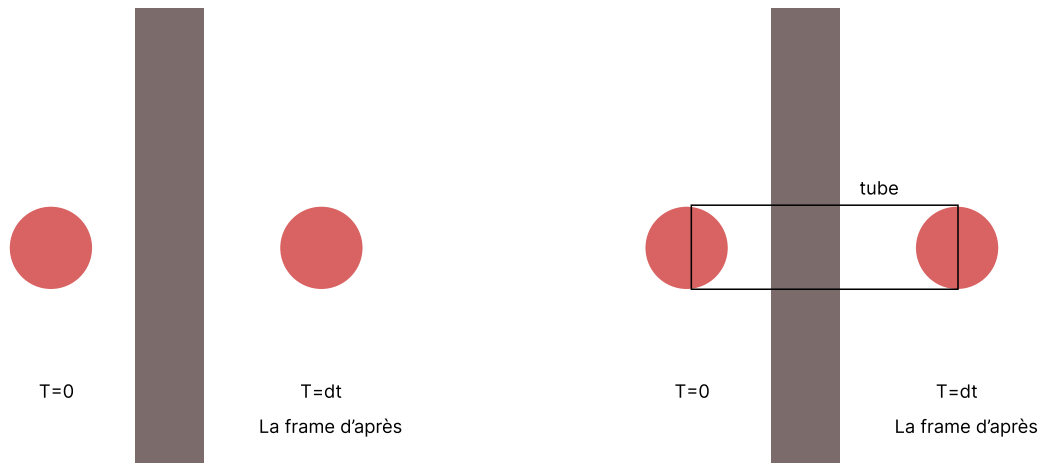


Figure 2: Tunneling : la balle rapide passe de l'autre côté du mur entre deux frames. La détection discrète (positions à t et $t + dt$) rate la collision. Le CCD teste la **trajectoire complète** entre t et $t + dt$.

La solution : CCD

Le **Continuous Collision Detection** teste la collision le long de la **trajectoire** entre t et $t + dt$, pas seulement aux positions discrètes :

- **Swept test** : on “balaie” la forme le long de son déplacement et on teste le volume couvert.
- **Raycast** : pour une sphère, on raycaste le centre le long de $\vec{v} \cdot dt$ avec un rayon élargi.
- **Conservative advancement** : on avance par petits pas adaptatifs jusqu'au premier contact.

Coût : le CCD est **plus cher** que la détection discrète. C'est pourquoi les moteurs l'activent **par objet** — uniquement pour les projectiles rapides, pas pour tous les objets.

CCD dans les moteurs.

- **PhysX / Unity** : `rigidbody.enableCcd = true` — pour les projectiles.
- **Rapier** : `RigidBodyDesc.dynamic().setCcdEnabled(true)`.
- **Godot** : propriété `continuous_cd` sur les `RigidBody`.
- **Notre moteur** : le sub-stepping de la Session 12 est une **rustine approximative** de CCD — on réduit dt pour que le déplacement par pas soit petit devant les obstacles.

Sleeping

Sleeping (mise en sommeil)

Si un objet est **immobile** (vitesse et vitesse angulaire sous un seuil pendant un certain temps), on arrête de le simuler. Il “dort” :

- Il ne consomme plus de temps de calcul.
- Il ne réagit plus aux forces tant qu'il dort.
- Il se **réveille** si un autre objet le touche, ou si on lui applique une force programmatique.

Pourquoi c'est essentiel : dans un niveau, 90 % des objets physiques sont au sol et immobiles. Sans sleeping, on simule inutilement des milliers de corps stables — le broad phase tourne, le solveur itère, et tout ça pour rien.

! Le piège du sleeping

Un objet endormi ne détecte plus les collisions **provoquées par lui**. Si un personnage marche sur un pont d'objets endormis, il faut que le premier contact **réveille** la chaîne. Les moteurs gèrent ça automatiquement — mais un objet au-dessus du sol qui “flotte” endormi est un bug classique quand on téléporte des objets sans les réveiller.

Les types de corps

Les trois types de rigid bodies

Tous les moteurs distinguent :

- **Dynamic** : subit les forces et les collisions. Masse finie. C'est l'objet physique standard.
- **Fixed (Static)** : ne bouge **jamais**. Masse infinie. Le sol, les murs, le décor.
- **Kinematic** : contrôlé **par le code** (position ou vitesse), ignore les forces, mais pousse les dynamic bodies. Plateformes mobiles, ascenseurs.

La distinction existe parce que simuler un mur comme un dynamic body de masse énorme coûte cher et cause de l'instabilité numérique. Un fixed body est **gratuit** et **parfaitement stable**.

Type	Subit les forces ?	Pousse les dyn. ?	Usage
Dynamic	Oui	Oui	Objets du jeu, débris, projectiles
Fixed	Non	Oui (mur)	Sol, murs, niveau
Kinematic	Non (code !)	Oui	Plateformes, ascenseurs, portes

Table 1: Les trois types de corps. Le kinematic est le plus subtil : le code contrôle sa trajectoire, mais les dynamic bodies réagissent quand il les pousse.

Les queries (raycast, shapecast)

Scene queries

Au-delà de la simulation, les moteurs offrent des **questions directes** sur l'état du monde :

- **Raycast** : “qu'est-ce que cette droite touche, et à quelle distance ?” — essentiel pour les tirs (hitscan), la ligne de vue, les IA.
- **Shapecast** : “si je déplace cette forme de A à B, que touche-t-elle en premier ?” — utilisé par les character controllers pour se déplacer sans traverser les murs.
- **Overlap** : “quels corps chevauchent ce volume ?” — zones de dégâts, pickups, déclencheurs.

Les queries **ne modifient pas** la simulation — elles l'interrogent.

? Le raycast, outil universel

Le personnage ne “marche” presque jamais avec un rigid body — il fait un **shapecast vers le bas** pour trouver le sol, puis le code déplace le personnage et résout les collisions manuellement. C'est pour ça que les moteurs ont des **CharacterBody** (Godot, Unity) : des corps non simulés qui utilisent les queries pour se déplacer de façon contrôlée. On les verra en Session 18.

Partie 3 : Le Paysage des Moteurs

Les moteurs natifs (C/C++)

PhysX — le standard historique

Développé par NVIDIA (originellement NovodeX, 2001). Open-source depuis 2016. C’est le moteur **par défaut** de Unity et Unreal Engine. On le trouve dans la plupart des AAA. Points forts : maturité, outils, GPU acceleration (pour la simulation massive). C’est un **standard de facto**.

Havok — le moteur premium

Moteur commercial fermé (Irlande, 2000). Utilisé par Half-Life 2, Zelda TotK, l’ensemble des grosses productions Sony/Microsoft. Réputé pour sa **stabilité** sur les empilements massifs et sa destruction avancée. Coûteux — réservé aux studios avec un budget.

Jolt Physics — le challenger moderne

Open-source (par Jorrit Rouwe, 2021), écrit en C++ moderne. Rapide, multithreadé, déterministe. Utilisé pour **Horizon Forbidden West** (démonstration de puissance), et il est devenu le moteur **par défaut** de Godot 4.4+. Le succès récent le plus notable du domaine.

Box2D — le roi du 2D

D’Erin Catto (2006), open-source. La référence 2D : Angry Birds, Terraria, et des centaines de jeux 2D. Le solveur de contraintes de Box2D (Sequential Impulse) est **le** solveur que tous les autres ont copié. Le moteur “maison 2D” de ce cours est architecturé comme un mini-Box2D.

Les moteurs web (JavaScript / WASM)

Rapier — le standard web moderne

Écrit en **Rust**, compilé en **WebAssembly**. Rapide, déterministe, API moderne. Le moteur **recommandé** pour Three.js. Développé par Dimforge. Nous allons l’utiliser aujourd’hui. Points forts : performance proche du natif, WASM (le code binaire tourne à pleine vitesse dans le navigateur), API propre.

Cannon.js / Ammo.js — la génération précédente

- **Cannon.js** : moteur en pur JavaScript (2011), simple mais lent, peu maintenu. Cannon-es est un fork maintenu.
- **Ammo.js** : portage Emscripten de Bullet Physics vers WASM. Performant mais API datée, gros bundle (2 Mo), moins pratique.

Ces moteurs ont précédé Rapier — aujourd’hui, pour un nouveau projet Three.js, **Rapier est le choix par défaut**.

Moteur	Langage	Licence	Usage notable
PhysX	C++	Open source	Unity/Unreal par défaut, AAA
Havok	C++	Commerciale	Half-Life 2, Zelda TotK
Jolt Physics	C++	Open source	Horizon FW, Godot 4.4+
Box2D	C++	Open source	Angry Birds, Terraria (2D)
Bullet	C++	Open source	Blender, GTA IV
Rapier	Rust→WASM	Open source	Web (Three.js), Bevy
Cannon-es	JavaScript	Open source	Three.js (simple, legacy)
Ammo.js	C++→WASM	Open source	Three.js (Bullet port)
Godot Physics	C++	Open source	Godot (Godot Physics/Jolt)

Table 2: Paysage des moteurs physiques. Notez Godot en bas : le moteur de jeu **intègre** son propre moteur physique (ou Jolt) — pas besoin d’en choisir un.

💡 Le retour de Jolt — une leçon d'humilité

Jolt a été écrit **par une seule personne** (Jorrit Rouwe) en quelques années, et il rivalise avec PhysX et Havok, développés par des équipes entières depuis 20 ans. Il a été adopté par Godot comme moteur par défaut. Preuve que le domaine n’est pas figé — et qu’une **compréhension profonde** des principes (les vôtres maintenant) permet de construire des outils de classe mondiale.

Partie 4 : Déterminisme

Déterminisme

Un moteur est **déterministe** si les mêmes inputs produisent **toujours exactement les mêmes outputs** — même sur des machines différentes, même après des millions de pas.

💬 Pourquoi c'est important

- **Multijoueur lockstep** : chaque client simule la même physique et n’échange que les inputs. Si la physique diverge d’un bit, les joueurs voient des mondes **différents** — le jeu casse. StarCraft II, Age of Empires IV fonctionnent comme ça.
- **Replays** : enregistrer seulement les inputs et rejouer la simulation requiert un moteur déterministe.
- **Tests de régression** : un bug de physique se reproduit exactement — on peut le déboguer.
- **Rollback netcode** : rembobiner la physique et rejouer (fighting games) exige le déterminisme.

Pourquoi c'est difficile

Le déterminisme se casse par :

- **Ordre des opérations** : si les paires de collision sont traitées dans un ordre différent (hash map, multithreading), les résultats divergent.
- **Précision flottante** : le format IEEE 754 est déterministe pour $+$, $-$, $\times$, $\div$ — mais pas pour sqrt , sin , cos (implémentations différentes selon le CPU/libm).
- **Multithreading** : deux threads qui traitent des corps en interaction dans un ordre non fixé $\rightarrow$ divergence.

Solutions : ordre de traitement **fixe et trié** (par handle, pas par adresse), maths à précision **fixée** (fixed-point ou entiers), fonctions trigonométriques **réimplémentées** (deterministic sin/cos), thread scheduling **constraint**.

⚠ En pratique

Rapier est déterministe **localement** (même machine, même binaire) mais pas **cross-platform** par défaut. Pour du lockstep multijoueur en production, on utilise des moteurs spécifiquement conçus pour ça (ou on fixe la précision manuellement). Godot **n'est pas** déterministe — les replays Godot enregistrent les états, pas les inputs.

Partie 5 : Architecture ECS

Entity-Component-System

L'architecture dominante des moteurs modernes (Unity DOTS, Bevy, Flecs, EnTT) :

- **Entity** : un simple **ID** (ex: `entity_42`). Pas de données, pas de logique. Juste une étiquette.
- **Component** : des **données pures** (ex: `Position`, `RigidBody`, `Mesh`). Un composant ne contient **aucune logique** — juste des champs.
- **System** : de la **logique pure** qui opère sur toutes les entités possédant certains composants (ex: `PhysicsSystem` traite tout ce qui a `RigidBody` + `Position`).

L'idée en code.

```
// Entity: juste un ID
const entity = world.createEntity();      // entity = 42

// Components: des données pures
world.addComponent(entity, Position, { x: 0, y: 10, z: 0 });
world.addComponent(entity, Velocity, { x: 0, y: 0, z: 0 });
world.addComponent(entity, RigidBody, { mass: 1.0 });

// System: de la logique qui traite toutes les entités
// possédant Position + Velocity
world.addSystem((dt) => {
    for (const e of world.query(Position, Velocity)) {
        e.position.x += e.velocity.x * dt;    // ...
    }
});
```

❗ Pourquoi ECS plutôt que l'héritage ?

L'approche orientée objet classique (`class Projectile extends PhysiqueObject extends GameObject`) crée des **hiérarchies rigides** : un objet "posable ET physique ET dégâts" force des contorsions d'héritage.

ECS compose : un baril = Position + Rigidbody + Mesh + Pickup. Une flèche = Position + Rigidbody + Mesh + Projectile. On **ajoute et enlève des composants à la volée** — un ennemi mort devient un ragdoll : on retire AI, on ajoute Rigidbody à ses membres.

Bonus majeur : les **systems** itèrent sur des tableaux de données contiguës (**cache-friendly**) — c'est ce qui rend les moteurs ECS si rapides.

💡 Le lien avec la physique

Rapier, en interne, ressemble à un ECS : chaque Rigidbody est une entité avec des composants (position, velocity, collider handles). Quand on écrit `world.step()`, un `PhysicsSystem` itère sur tous les corps. Vous avez **déjà** écrit ce code — votre boucle `updatePhysics(dtFrame)` de la Session 12 est un `PhysicsSystem` !

Partie 6 : Pratique — Three.js + Rapier

💡 L'exemple du jour

Ouvrez `session13_rapier.html` via Live Server. C'est une scène Three.js classique — sol, pyramide de cubes, sphère témoin — où toute la physique est déléguée à **Rapier**. Comparez avec nos sessions précédentes : **il n'y a plus une seule ligne d'intégration, de collision ou d'impulsion**. On crée des corps, on les laisse vivre, et on synchronise les meshes sur les positions du moteur. Cette partie parcourt le projet **fichier par fichier, composant par composant**.

Structure du projet

Les fichiers de l'exemple

```
examples/  
├─ session13_rapier.html  ← la page web : importmap, HUD, style  
├─ session13_rapier.js    ← tout le code (rendu + physique)  
└─ textures/  
    └─ grid.png           ← texture du sol (réutilisée du cours)
```

- **session13_rapier.html** : la coquille. Contient le `#hud` (compteur de corps/endormis), l'**importmap** qui mappe "three" et "rapier" vers des CDN, et charge `session13_rapier.js` comme module.
- **session13_rapier.js** : le cœur. Un seul fichier : la scène Three.js, le monde Rapier, et le **lien** entre les deux.

💡 L'importmap — le seul changement côté HTML

Les modules ES ne savent pas résoudre les noms nus (`import ... from 'three'`). L'**importmap** déclare : "three" → CDN unpkg, "jsm/" → les addons Three.js, et — nouveauté de cette session

— "rapier" → le CDN skypack du package @dimforge/rapier3d-compat. Le JS fait ensuite simplement `import RAPIER from 'rapier'`.

Composant 1 — Charger le moteur (WASM)

Rapier est un binaire, pas du JavaScript

Rapier est écrit en **Rust** et compilé en **WebAssembly** (WASM) — c'est ce qui lui donne ses performances proches du natif. Le package -compat embarque le binaire en base64. Conséquence : le chargement est **asynchrone** — impossible de créer quoi que ce soit avant `RAPIER.init()`. Toute la fonction `init()` de l'exemple est donc `async` :

```
import RAPIER from 'rapier';          // résolu par l'importmap

async function init() {
  await RAPIER.init();                 // ← le WASM se décode ici
  // seulement APRÈS : world, bodies, colliders...
}
```

Composant 2 — Le monde physique

L'équivalent de notre « univers » maison

```
const world = new RAPIER.World({ x: 0, y: -9.81, z: 0 });
world.timestep = FIXED_DT;           // 1/60 — pas fixe !
```

Le monde est le **registre** de tous les corps + la gravité + le pas de temps. C'est l'équivalent de notre tableau `planets` + la constante `G` des sessions précédentes — mais il gère aussi la broad phase, la broad SAP, le solveur, le sleeping... tout ce qu'on a écrit à la main.

Composant 3 — Un corps FIXE : le sol

RigidBodyDesc.fixed() + ColliderDesc.cuboid()

```
// --- Rendu : le mesh avec la texture grille ---
const mesh = new THREE.Mesh(
  new THREE.BoxGeometry(60, 0.2, 60), gridMaterial);

// --- Physique : corps FIXE (masse infinie, jamais simulé) ---
const body = world.createRigidBody(
  RAPIER.RigidBodyDesc.fixed().setTranslation(0, -0.1, 0));
world.createCollider(
  RAPIER.ColliderDesc.cuboid(30, 0.1, 30) // ← DEMI-tailles !
  .setFriction(params.friction)
  .setRestitution(params.restitution),
  body);
```

Deux objets distincts : le **rigid body** (le comportement — fixe, dynamique...) et le **collider** (la forme — cuboïde, boule...). Un body peut porter plusieurs colliders ; un collider sans body n'existe pas.

Attention au piège classique : `ColliderDesc.cuboid(hx, hy, hz)` prend des **demi-dimensions** (half-extents), alors que `BoxGeometry(w, h, d)` de Three.js prend des dimensions **complètes**. Un cube de 1 m = `BoxGeometry(1,1,1)` côté rendu, `cuboid(0.5, 0.5, 0.5)` côté physique.

setFriction et setRestitution correspondent exactement à la friction et à la restitution qu'on a codées à la main en Session 6.

Composant 4 — Des corps DYNAMIQUES : la pyramide

Le pattern « pairs » — le lien rendu ↔ physique

```
const pairs = []; // { mesh, body } pour CHAQUE objet physique

// Dans createStack() — une pyramide « bowling » :
for (let row = 0; row < 5; row++) {
  for (let i = 0; i < 5 - row; i++) {
    // Rendu
    const mesh = new THREE.Mesh(boxGeo, cubeMaterial);
    scene.add(mesh);
    // Physique : corps DYNAMIQUE
    const body = world.createRigidBody(
      RAPIER.RigidBodyDesc.dynamic()
        .setTranslation(x, 0.5 + row * 1.05, 0));
    world.createCollider(
      RAPIER.ColliderDesc.cuboid(0.5, 0.5, 0.5), body);
    // Le lien croisé — c'est LUI qu'on itère dans animate()
    pairs.push({ mesh, body });
  }
}
```

Rapier ne sait rien de Three.js, et Three.js ne sait rien de Rapier. Chaque objet physique de la scène existe donc **en double** : un mesh (ce qu'on voit) et un body (ce qui est simulé). Le tableau pairs maintient la correspondance — c'est le **cœur de toute intégration** moteur physique + moteur de rendu.

Composant 5 — La boucle : accumulateur, step, synchronisation

Le fixed timestep de la Session 5, appliqué à Rapier

```
const FIXED_DT = 1 / 60;
let accumulator = 0;

function animate() {
  const dt = Math.min(clock.getDelta(), 0.1);
  accumulator += dt;
  while (accumulator >= FIXED_DT) {
    world.step(); // ← TOUTE la physique ici
    accumulator -= FIXED_DT;
  }
  // — Synchronisation : rendu ← physique —
  for (const { mesh, body } of pairs) {
    const t = body.translation(); // { x, y, z }
    const r = body.rotation(); // quaternion
    mesh.position.set(t.x, t.y, t.z);
    mesh.quaternion.set(r.x, r.y, r.z, r.w);
  }
  renderer.render(scene, camera);
}
```

La physique tourne à **pas fixe** (60 Hz), indépendamment du framerate — sinon la simulation serait 2× plus rapide sur un écran 120 Hz. Si une frame dure 32 ms, la boucle while fait **deux pas**. Puis on **recopie** les positions et rotations du moteur vers les meshes.

Règle d'or : la physique est la **source de vérité**. On ne déplace **jamais** le mesh directement — on pousse le body (force, impulsion), et le mesh suit à la frame suivante.

Composant 6 — Le boulet : impulsion + CCD

applyImpulse + setCcdEnabled — la démo tunneling

```
function fireCannonball(origin, dir) {
  // 1. Rendu
  const mesh = new THREE.Mesh(
    new THREE.SphereGeometry(0.45, 24, 24),
    new THREE.MeshStandardMaterial({ color: 0x2c3e50, metalness: 0.9,
roughness: 0.25 }));
  scene.add(mesh);

  // 2. Physique : CCD activé, collider lourd
  const body = world.createRigidBody(
    RAPIER.RigidBodyDesc.dynamic()
      .setTranslation(origin.x, origin.y, origin.z)
      .setCcdEnabled(params.ccd)); // ← CCD ici !
  world.createCollider(
    RAPIER.ColliderDesc.ball(0.45)
      .setDensity(10), // lourd — 10× la densité par défaut
    body);

  // 3. Impulsion : Session 6 en une ligne
  body.applyImpulse(
    { x: dir.x * params.cannonSpeed, y: dir.y * params.cannonSpeed, z: dir.z *
params.cannonSpeed },
    true); // true = réveille le corps
  pairs.push({ mesh, body });
}
```

Le boulet est le cas d'école du **tunneling** (Partie 2) : à 120 unités/s, il avance de **2 unités par pas** — plus qu'un cube entier. Désactivez le CCD dans la GUI et tirez à haute vitesse : il **traverse** la pyramide sans la toucher. Réactivez-le : il la démolit. `applyImpulse(vec, true)` est l'exact équivalent de l'impulsion $\vec{J} = m \cdot \Delta\vec{v}$ de la Session 6 — le moteur la convertit en changement de vitesse.

Composant 7 — Le HUD : lire le sleeping

isSleeping() — la preuve que le moteur optimise

```
function updateHUD() {
  let sleeping = 0, dynamic = 0;
  for (const { body } of pairs) {
    if (body.isDynamic()) dynamic++;
    if (body.isSleeping()) sleeping++;
  }
  hud.textContent = `Corps : ${dynamic} | Endormis : ${sleeping}`;
}
```

Laissez la scène se stabiliser : le compteur d'**endormis** grimpe vers le total. Le moteur a **arrêté de simuler** les cubes immobiles (Partie 2) — zéro coût CPU pour eux. Touchez la pyramide avec un boulet : les cubes touchés **se réveillent** en cascade.

Le piège du sleeping : quand la GUI change la gravité, il faut **réveiller tout le monde** (`body.wakeUp()` dans l'exemple) — sinon les corps endormis continuent d'ignorer la nouvelle gravité !

Composant 8 — Le ménage

removeRigidBody + reset

```
function removePair(pair) {
    scene.remove(pair.mesh);
    pair.mesh.geometry.dispose();
    pair.mesh.material.dispose();
    world.removeRigidBody(pair.body); // supprime aussi ses colliders
    pairs = pairs.filter(p => p !== pair);
}
```

Comme toujours : on libère la mémoire GPU (geometry/material) et on retire le body du monde. Le bouton **Reset** de la GUI vide `pairs` puis reconstruit la pyramide. Les boulets sont plafonnés à 12 pour la performance — un vrai moteur gère des **milliers** de corps, mais chaque `world.step()` a un coût.

Ce que Rapier fait pour nous — et que nous avons fait à la main

- **L'intégration** : un solveur TGS (Tiered Gulick Solver, un solveur d'impulsions itératif supérieur à notre Euler).
- **La détection** : broad phase SAP + narrow phase GJK/EPA — **exactement** ce qu'on a vu en Sessions 7 et 11.
- **La réponse** : impulsions + correction de position (Session 6), mais avec des itérations multiples et de la stabilisation (Baumgarte).
- **Le sleeping, le CCD, les joints** : tout y est.

La différence : 20 ans de recherche et d'optimisation condensés dans un binaire WASM de 1,5 Mo.

Lancer la boule de destruction

Dans l'exemple, cliquez pour lancer un boulet vers la pyramide (le raycast part de la caméra vers le pixel cliqué). Montez la **vitesse** à 120 dans la GUI, désactivez le **CCD**, tirez : le boulet traverse les cubes — le tunneling, en live. Réactivez le CCD, tirez encore : la pyramide explose. C'est la démonstration la plus convaincante de l'utilité du CCD.

Partie 7 : Pratique — Godot

Godot — le moteur intégré

Godot illustre l'autre approche : le moteur de jeu **intègre** la physique (Godot Physics ou Jolt). Pas de bibliothèque à choisir, pas de synchronisation à écrire — les nœuds physiques sont à la fois **rendus** et **simulés**. Le projet demo-physics reproduit la même scène que la version Rapier :

pyramide, boulet, HUD sleeping — pour comparer les deux philosophies. Cette partie détaille l'arborescence du projet et le rôle de chaque composant GDScript.

Structure du projet

Les fichiers de `demo-physics`

```
demo-physics/
├─ project.godot          ← moteur Jolt, scène principale, autoloads
├─ scenes/
│   └─ session13_demo.tscn ← racine Node3D + le script attaché
├─ scripts/
│   └─ session13_demo.gd   ← tout le code (construit la scène au runtime)
├─ textures/
│   └─ grid.png            ← texture du sol (la même que Three.js !)
└─ addons/
    └─ godot_mcp_toolkit/ ← outils pédagogiques (non requis pour jouer)
```

Deux différences majeures avec le projet Three.js :

- **Pas de HTML ni d'importmap** — Godot gère lui-même le chargement des ressources (res://).
- **La scène .tscn est minimale** : une racine Node3D avec le script attaché. Tout le reste (caméra, lumière, sol, pyramide) est construit **par code** dans `_ready()`, pour que tout tienne dans un seul fichier comparable à la version Rapier.

💡 Le `project.godot` — trois lignes qui comptent

```
[physics]
3d/physics_engine="Jolt Physics"    ← active Jolt (Partie 3)

[application]
run/main_scene="res://scenes/session13_demo.tscn"
```

On **choisit** le moteur physique du projet comme un paramètre. Jolt remplace le moteur par défaut depuis la 4.4. Même architecture de cours (broad phase, solveur), autre implémentation.

Composant 1 — `_ready()` : construire la scène par code

La méthode d'initialisation

```
extends Node3D

var _cannon_speed := 60.0
var _ccd_enabled := true
var _camera: Camera3D
var _hud: Label
var _dynamic_bodies: Array[RigidBody3D] = []

func _ready() -> void:
    _setup_camera()
    _setup_light()
    _setup_ground()
    _setup_hud()
    _spawn_stack()
```

`_ready()` est l'équivalent de la `init()` du JS : elle s'exécute une fois au démarrage. Elle appelle cinq fonctions qui créent les cinq sous-systèmes : caméra, lumière, sol, HUD, empilement.
Aucune scène n'est préfabriquée dans l'éditeur — c'est un choix pédagogique pour montrer que Godot permet de tout faire par code.

Composant 2 — Le sol : `StaticBody3D`

Corps FIXE avec enfant `'CollisionShape3D'` + `'MeshInstance3D'`

```
func _setup_ground() -> void:
    var ground := StaticBody3D.new()
    ground.position = Vector3(0, -0.1, 0)

    # --- Forme de collision ---
    var col := CollisionShape3D.new()
    var box := BoxShape3D.new()
    box.size = Vector3(60, 0.2, 60)    # ← taille COMPLÈTE
    col.shape = box
    ground.add_child(col)

    # --- Rendu ---
    var mesh := MeshInstance3D.new()
    var box_mesh := BoxMesh.new()
    box_mesh.size = Vector3(60, 0.2, 60)
    mesh.material_override = gridMaterial
    mesh.mesh = box_mesh
    ground.add_child(mesh)

    add_child(ground)
```

Un `StaticBody3D` est le correspondant du `RigidBodyDesc.fixed()` de Rapier. Deux enfants : `CollisionShape3D` (la physique) et `MeshInstance3D` (le rendu). Ils sont **sous le même nœud** : Godot sait qu'il doit synchroniser la forme et le mesh.

Attention au piège inverse de Rapier : dans Godot, `BoxShape3D.size` et `BoxMesh.size` prennent la taille **complète** du cube (pas les demi-tailles). C'est plus intuitif, mais il faut le savoir pour ne pas faire un sol de 30 m au lieu de 60 m.

Composant 3 — La pyramide : `RigidBody3D` par code

Chaque cube est un `RigidBody3D` avec deux enfants

```
func _spawn_stack() -> void:
    var cube_mesh := BoxMesh.new()
    cube_mesh.size = Vector3.ONE

    for row: int in 5:
        var count := 5 - row
        for i in count:
            var cube := RigidBody3D.new()
            cube.position = Vector3((i - (count - 1) / 2.0) * 1.05, 0.5 + row *
1.05, 0)

            var col := CollisionShape3D.new()
            var shape := BoxShape3D.new()
```

```

shape.size = Vector3.ONE
col.shape = shape
cube.add_child(col)

var mesh := MeshInstance3D.new()
var mat := StandardMaterial3D.new()
mat.albedo_color = CUBE_COLORS[row % CUBE_COLORS.size()]
mesh.material_override = mat
mesh.mesh = cube_mesh
cube.add_child(mesh)

add_child(cube)
_dynamic_bodies.append(cube)

```

Le RigidBody3D est à la fois le corps dynamique de Rapier et le **conteneur de la hiérarchie**. On lui ajoute un CollisionShape3D pour la physique et un MeshInstance3D pour le rendu. Godot **oriente automatiquement** le mesh quand le body bouge : **aucune boucle de synchronisation** à écrire.

_dynamic_bodies est le pendant du tableau pairs de la version Rapier, mais il ne contient que les corps — le lien avec le mesh est implicite (enfant de chaque body).

Composant 4 — Le tir : rayon caméra + CCD + vitesse

`\_fire\_cannonball()` — le même tunneling, la même démo

```

func _fire_cannonball(screen_pos: Vector2) -> void:
    var from := _camera.project_ray_origin(screen_pos)
    var dir := _camera.project_ray_normal(screen_pos)

    var ball := _make_ball(from + dir * 2.0, 0.45, Color(0.17, 0.24, 0.32), 5.0)
    ball.continuous_cd = _ccd_enabled      # ← CCD ici (propriété du nœud)
    ball.linear_velocity = dir * _cannon_speed # ← vitesse directe
    _ball_count += 1
    _dynamic_bodies.append(ball)

```

La caméra fournit project_ray_origin() et project_ray_normal() — le raycast écran → monde. On crée un RigidBody3D à partir de _make_ball(), on active continuous_cd, et on donne une vitesse initiale. Pas d'appel à apply_impulse ici : **linear_velocity est équivalent** quand la vitesse initiale est connue. C'est l'équivalent du setLinvel() / applyImpulse() de Rapier.

Composant 5 — Les entrées : _unhandled_input

Un seul callback pour tout

```

func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventMouseButton \
        and event.pressed and event.button_index == MOUSE_BUTTON_LEFT:
        _fire_cannonball(event.position)
    elif event is InputEventKey and event.pressed and not event.echo:
        match event.keycode:
            KEY_R: _reset()
            KEY_C: _ccd_enabled = not _ccd_enabled
            KEY_UP: _cannon_speed = minf(_cannon_speed + 10.0, 150.0)
            KEY_DOWN: _cannon_speed = maxf(_cannon_speed - 10.0, 10.0)

```

Un seul gestionnaire remplace le `setupClick()` de Rapier + les écouteurs de clavier. Godot propose `InputEvent` typés : `InputEventMouseButton`, `InputEventKey`. Le `match` GDScript remplace le `switch/if-else`. La vitesse et le CCD sont des variables membres utilisées lors du **prochain** tir.

Composant 6 — Le HUD sleeping

`_process` lit le moteur à chaque frame

```
func _process(_delta: float) -> void:
    var sleeping := 0
    for body in _dynamic_bodies:
        if body.is_sleeping():
            sleeping += 1
    _hud.text = "Corps dynamiques : %d | Endormis : %d\n" % [ ... ]
    _hud.text += "Vitesse du boulet : %d (↑/↓) | CCD : %s (C)\n" % [ ... ]
```

`_process(delta)` est appelé à **chaque frame rendue** (framerate variable). C'est le bon endroit pour mettre à jour le HUD. Le moteur physique tourne dans `_physics_process(delta)` (60 Hz fixe), mais on n'en a pas besoin ici : Godot gère le `step()` en interne.

Composant 7 — Pas de synchronisation, pas de `step()`

Ce que Godot cache — et pourquoi c'est plus rapide à prototyper

Dans le projet Godot, **aucune boucle** ne copie `body.position` vers `mesh.position`. Le `RigidBody3D` est le parent, le `CollisionShape3D` et le `MeshInstance3D` sont des enfants : Godot met à jour la transformation de toute la hiérarchie quand le moteur avance le body.

Le `world.step()` est lui aussi caché. C'est `_physics_process(delta)` et les paramètres `physics/physics_ticks_per_second` qui font le travail, exactement comme l'accumulateur de la Session 5.

La contrepartie : on a **moins de contrôle** sur le moment exact du pas physique. Pour des jeux classiques c'est un avantage ; pour des réseaux à pas prédictifs ou des simulations très spécifiques, c'est une contrainte.

Tableau comparatif : même concept, deux API

Concept	Three.js + Rapier (JS)	Godot 4 (GDScript)
Monde physique	<code>new RAPIER.World(gravity)</code>	Automatique — <code>ProjectSettings / Jolt</code>
Pas fixe	<code>while(accumulator >= FIXED_DT) world.step()</code>	<code>_physics_process(delta)</code> — 60 Hz
Corps fixe	<code>RigidBodyDesc.fixed()</code>	<code>StaticBody3D</code>
Corps dynamique	<code>RigidBodyDesc.dynamic()</code>	<code>RigidBody3D</code>
Forme cubique	<code>ColliderDesc.cuboid(0.5, 0.5, 0.5)</code> (demi-tailles)	<code>BoxShape3D.new(); shape.size = Vector3.ONE</code> (taille complète)
Forme sphérique	<code>ColliderDesc.ball(0.45)</code>	<code>SphereShape3D.new(); shape.radius = 0.45</code>
Synchronisation	Manuelle : <code>mesh.position.copy(body.translation())</code>	Automatique : nœud enfant
CCD	<code>body.setCcdEnabled(true)</code>	<code>ball.continuous_cd = true</code>
Vitesse initiale	<code>body.setLinvel(v)</code>	<code>ball.linear_velocity = v</code>
Impulsion	<code>body.applyImpulse(v, true)</code>	<code>ball.apply_impulse(v)</code>
Sleeping	<code>body.isSleeping()</code>	<code>body.is_sleeping()</code>
Gravité	<code>world.gravity = { ... }</code>	<code>ProjectSettings</code> ou <code>Area3D</code>
Suppression	<code>world.removeRigidBody(body) + ménage mesh</code>	<code>body.queue_free()</code>
Téléportation	À éviter — utiliser forces/impulsions	À éviter — préférer <code>_integrate_forces()</code>

Table 3: Même physique, deux façons de la piloter. Les concepts sont identiques, seule l'API change.

Godot : physique à pas fixe

Comme notre Session 5 : Godot simule la physique à **60 Hz fixes** (`physics/physics_ticks_per_second`), indépendamment du framerate. La logique physique va dans `_physics_process(delta)`, la logique de rendu dans `_process(delta)`. Si la frame a duré 32 ms, la physique rattrape en faisant **2 pas**. C'est exactement le pattern **fixed timestep + accumulateur** qu'on a implémenté — sauf qu'il est invisible.

🔗 Jolt intégré dans Godot 4.4+

Depuis Godot 4.4, le moteur Jolt Physics (Partie 3) est disponible comme option de projet : `physics/3d/physics_engine = "Jolt Physics"`. Le projet `demo-physics` l'utilise. Mieux : stabilité des empilements, performance multithreadée — même architecture, meilleures maths.

Partie 8 : Choisir son moteur — le guide pratique

Contexte	Choix recommandé	Pourquoi
Jeu web Three.js	Rapier	WASM rapide, API moderne, le standard
Jeu web 2D simple	Rapier 2D ou Planck.js	Rapide et simple
Jeu Godot	Intégré (Jolt)	Zéro intégration, outils inclus
Jeu Unreal	Chaos (intégré)	Intégré + destruction
Jeu Unity	PhysX (défaut)	Intégré, mature
Simulation massive	PhysX GPU / Jolt	Multithread, GPU
Multijoueur lockstep	Moteur déterministe dédié	Floats cross-platform

Table 4: Guide de choix rapide. Dans tous les cas : les concepts sont les mêmes, seule l'API change.

Synthèse

Ce qu'il faut retenir

- **Un moteur physique commercial** économise des années de R&D — mais sans comprendre les principes (intégration, collisions, solveur), on ne peut pas le déboguer ni le pousser dans ses retranchements.
- Tous les moteurs partagent les mêmes concepts : **rigid bodies** (dynamic/fixed/kinematic), **colliders**, **CCD** anti-tunneling, **sleeping**, **queries** (raycast).
- **Le pattern d'intégration** est universel : créer le monde, créer mesh + body par objet, `step()`, synchroniser. La physique est la source de vérité.
- **Le déterminisme** (mêmes inputs → mêmes outputs) est essentiel pour le multijoueur lockstep et les replays, mais difficile (ordre des opérations, flottants, threads).
- **L'ECS** (Entity-Component-System) est l'architecture des moteurs modernes : composition par données, logique dans les systems, cache-friendly.
- **Rapier** est le standard pour Three.js ; **Godot** intègre sa propre physique (Jolt) — deux philosophies : bibliothèque vs moteur intégré.

La suite

Session 14 : les **corps rigides en détail** — boîtes de collision, moments d'inertie, et comment les moteurs simulent la rotation des solides. Puis Joints et moteurs (S15), contraintes et IK (S16), et le TP véhicules (S17) — où tout ce qu'on voit aujourd'hui servira en pratique.